



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Selected Topics in Cryptography

Reporte Práctica 04:

“ECDSA”

Alumno:

Sigala Morales Said

2022630413

Grupo: 7CM4

Maestro: Sandra Diaz Santiago

Fecha de realización: 19 de marzo de 2025

Fecha de entrega: 19 de marzo de 2025



PROGRAMMING EXERCISES

Use a file to store the public parameters of the protocol ECDSA: a prime number $p > 255$, the parameters of an elliptic curve $ab \mathbb{Z}_p$, a generator point $G \in E(ab)$ and the order of G , q . These parameters **MUST NOT** be generated at random. Please consider that any user must have access to this file, before generating the key pair or before generating the signature. You are free to define the structure of your file and the format of your file

```
parametros.txt
1  a=17
2  b=28
3  p=257
4  g= 2,29,1
5  d=5
6
```

Esta fue el formato que elegí, los primeros 3 son los parámetros de la curva, el punto g es el punto generador y d es el escalar por el cual se va a multiplicar el generador, aunque no se bien si hacerlo aquí o pedirlo durante la ejecución del código debido a que debe ser un parámetro privado y mantenerlo en los parámetros públicos no resulta lo mas seguro.



Design and implement a function to generate a pair of keys for ECDSA, once that the public parameters were given. The public key must be stored in a text file as a tuple (pab qGB), where GB $E(ab)$.

```
1 def generate_key(a, b, p, d, g):
2
3     x = g[0] % p
4     y = g[1] % p
5     z = g[2] % p
6     G = (x, y, z)
7
8     generador = SumaDoblado.show_generator(a, b, p, G, SumaDoblado.point_add, SumaDoblado.point_double)
9     print(f"El punto G es generador: {generador}")
10
11     puntos_en_curva = SumaDoblado.find_points_on_curve(a, b, p)
12     cardinalidad = len(puntos_en_curva) + 1
13     print(f"Cardinalidad de la curva: {cardinalidad}")
14     if d < 1 or d > cardinalidad:
15         raise ValueError("El valor de d debe estar entre 1 y la cardinalidad de la curva")
16
17     B = SumaDoblado.point_mul(a, b, p, d, G, SumaDoblado.point_add, SumaDoblado.point_double)
18     Kpub = (p, a, b, cardinalidad, G, B)
19     Kpriv = d
20
21     return Kpub, Kpriv
```

Descripción de la función:

Primero se aplica modulo a los valores del punto y genera el nuevo punto. La verificación de que sea una curva no singular y que el primo sea mayor a 255 se hacen en la función que lee los parámetros del punto.

Posterior se llama a la función que muestra si es generador, esto va devolver si es punto es un generador o si simplemente genera un subgrupo de puntos.

Una vez con eso tenemos que saber cual es el orden de q, este será la cardinalidad del punto, por eso se llama a la función que genera los puntos y luego calcula su cardinalidad sabiendo la longitud del arreglo de los puntos y le suma 1 por el punto al infinito.

Posterior verifica que d sea mayor a 1 y menor al orden q, de no ser así devolverá una excepción.

Ahora con el valor de d comprobado se hace la multiplicación escalar para calcular β . Con esto calculado ponemos generar las claves que es lo que hacemos y las devuelve la función.



EJEMPLO DE APLICACIÓN

```
PS C:\Users\said\Documents\ESCOM\8voSemestre\CRIPTOGRAFIA SELECTED TOPICS IN CRYPTOGRAPHY\Practicas\Practica02> & C:/Users/said/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/said/Documents/ESCOM/8voSemestre/CRIPTOGRAFIA SELECTED TOPICS IN CRYPTOGRAPHY/Practicas/Practica02/practica04.py"
Parámetros leídos: a=17, b=28, p=257, d=88, g=(2, 29, 1)
El orden del punto es: 88
El punto G es generador: False
Cardinalidad de la curva: 264
Clave pública: (257, 17, 28, 264, (2, 29, 1), (0, 1, 0))
Clave privada: 88
PS C:\Users\said\Documents\ESCOM\8voSemestre\CRIPTOGRAFIA SELECTED TOPICS IN CRYPTOGRAPHY\Practicas\Practica02> █
```